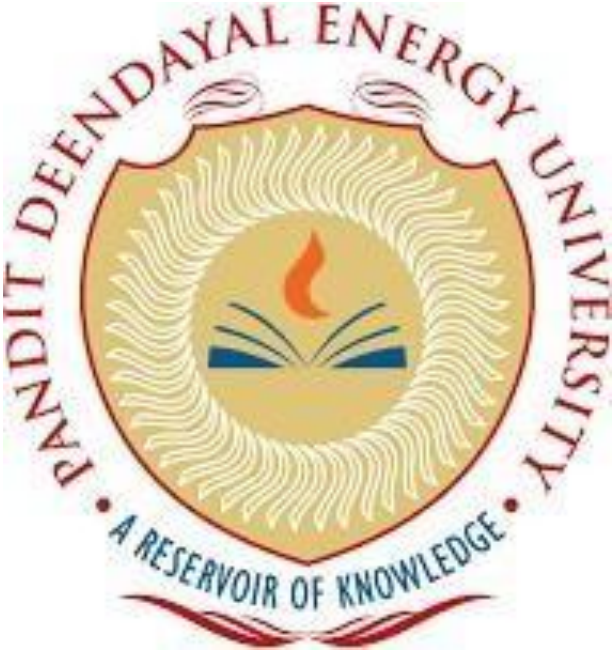
Electronics System Design Lab

Project Report on

ESP32-Based Two-Way LoRa Communication System with Real-Time Web Interface and OLED Monitoring



Group Members:
Preet Desai– 23BEC020
Dhruvi Singh – 23BEC028

Submitted To:
Dr. Debajyoti Mondal

Table of Contents

1. Introduction
2. Literature Review
3. Working Principle
   - 3.1 Message Transmission
   - 3.2 Over-the-Air Communication
   - 3.3 Message Reception
   - 3.4 ACK-Based Reliability
   - 3.5 OLED Monitoring System
4. Design Flow
   - 4.1 Hardware Design Flow
   - 4.2 Software Design Flow
   - 4.3 Web Interface Implementation
   - 4.4 Communication Protocol
5. Circuit Diagram
   - 5.1 Pin Connection Table
   - 5.2 Hardware Setup
6. Working of the Model
   - 6.1 Transmitter Node Operation
   - 6.2 Receiver Node Operation
   - 6.3 System Architecture
   - 6.4 Data Flow
   - 6.5 Web Server and Browser Interface
7. Results
   - 7.1 Communication Performance
   - 7.2 RSSI Monitoring
   - 7.3 Web Interface Performance
   - 7.4 OLED Monitoring Results
   - 7.5 ACK-Based Delivery Verification
8. Applications
   - 8.1 Emergency Communication
   - 8.2 Disaster Management
   - 8.3 Remote Area Communication
   - 8.4 IoT Messaging Systems
   - 8.5 Educational Wireless Research
9. Future Scope and Conclusion
   - 9.1 Future Scope
   - 9.2 Conclusion
10. References

Appendix

- ESP32 ALPHA Node Arduino IDE Code
- ESP32 DELTA Node Arduino IDE Code

1. Introduction

Wireless communication systems play a major role in modern Internet of Things (IoT), remote monitoring, and low-power long-range communication applications. Among various wireless technologies, LoRa (Long Range) communication has gained significant attention due to its ability to provide low-power communication over long distances while operating in license-free ISM bands.

This project presents a real-time two-way LoRa communication system implemented using ESP32 microcontrollers and SX1278 LoRa transceiver modules. The system establishes bidirectional communication between wireless nodes using the LoRa physical layer operating at 433 MHz. In addition to LoRa communication, the ESP32 creates a Wi-Fi Access Point and hosts a responsive web-based messaging interface that allows users to send and receive messages directly from a mobile phone or laptop browser.

The proposed system integrates multiple technologies including LoRa communication, embedded web servers, OLED display interfacing, packet acknowledgement mechanisms, and real-time signal monitoring. Messages transmitted between nodes are acknowledged using an ACK-based delivery confirmation protocol to improve communication reliability. An SSD1306 OLED display provides real-time monitoring of message status, signal strength (RSSI), and communication activity.

The system demonstrates a low-cost and portable communication platform suitable for remote messaging, emergency communication systems, IoT applications, and wireless experimentation. The project also highlights practical implementation of PHY-layer LoRa communication combined with modern browser-based interfaces.

2. Literature Review

Existing wireless communication systems based on LoRa technology primarily focus on sensor data transmission, LoRaWAN infrastructure, and long-range IoT deployments. Recent research has explored low-power communication systems, LoRa-based telemetry, and adaptive wireless communication protocols.

Most existing systems either focus on:

- Sensor-based one-way communication
- LoRaWAN cloud integration
- PHY-layer signal analysis
- SDR-based experimental platforms

However, limited work has been carried out on low-cost real-time two-way LoRa messaging systems integrated with embedded web interfaces and acknowledgement-based communication.

The proposed system introduces the following unique features:

- Real-time two-way LoRa communication
- Mobile-optimized browser-based chat interface
- ACK-based reliable message delivery
- OLED-based live system monitoring
- Integrated ESP32 Wi-Fi Access Point
- RSSI monitoring and communication diagnostics
- Standalone communication without internet dependency

The system provides a compact and cost-effective communication platform suitable for educational, experimental, and emergency communication applications.

Additionally, LoRa technology is widely used for low-power and long-range wireless communication in IoT applications. ESP32-based LoRa systems are popular due to their low cost, flexibility, and ease of implementation.

However, many existing systems lack real-time two-way communication, embedded web interfaces, and standalone operation without internet connectivity. The proposed system addresses these limitations by combining LoRa messaging, ACK-based communication, OLED monitoring, and an ESP32 Wi-Fi Access Point in a single compact platform.

3. Working Principle

The system operates using two ESP32 nodes equipped with SX1278 LoRa transceiver modules. One node acts as ALPHA while the second node acts as DELTA. Communication takes place through LoRa packets transmitted over the 433 MHz frequency band.

3.1 Message Transmission

The user connects a smartphone or laptop to the ESP32 Wi-Fi Access Point created by the transmitter node. A web-based messaging interface hosted by the ESP32 allows users to type and send messages.

When a message is entered:

1. The browser sends an HTTP request to the ESP32 web server.
2. The ESP32 processes the request.
3. The message is packed into a LoRa packet.
4. The SX1278 module transmits the packet wirelessly.

The transmitted packet follows the format:
ALPHA:<message>

3.2 Over-the-Air Communication

The LoRa transceiver sends packets through the wireless channel using Chirp Spread Spectrum (CSS) modulation.

During propagation, the signal may experience:

- Path loss
- Interference
- Noise
- Multipath fading

The receiver node continuously listens for incoming packets using LoRa.parsePacket().

3.3 Message Reception

When a valid packet is received:

- The receiver extracts the message.
- RSSI values are measured.
- The message is displayed on the OLED screen.
- The message is stored in memory.
- An acknowledgement packet is transmitted back.

The acknowledgement format is:
ACK:DELTA:<message>

3.4 ACK-Based Reliability

After transmitting a message, the sender waits for an acknowledgement for 2.5 seconds.

If the ACK is received:

- Message status changes to Delivered.
- RSSI value is updated.
- OLED status is refreshed.

This mechanism improves communication reliability and confirms successful packet delivery.

3.5 OLED Monitoring System

An SSD1306 OLED display connected through I2C provides real-time diagnostics including:

- Total messages
- RSSI signal strength
- Sent/received status
- ACK confirmation
- Recent communication logs

3.6 Web Interface Functionality

The embedded web interface provides a simple and user-friendly communication platform accessible through any mobile or desktop browser. The interface allows users to send messages, view received messages, and monitor delivery status in real time. JavaScript-based auto-refresh functionality ensures smooth message updates without requiring page reloads.

3.7 System Advantages

The overall system operates without requiring internet connectivity or external servers, making it suitable for remote-area communication and emergency situations. The combination of LoRa technology, ESP32 processing, OLED monitoring, and ACK-based messaging creates a reliable, portable, and low-cost wireless communication solution.

4. Design Flow

4.1 Hardware Design Flow

Step 1- Component Selection
The system uses:

- ESP32 NodeMCU microcontroller
- SX1278 LoRa transceiver module
- SSD1306 OLED display
- Mobile browser interface

The ESP32 was selected due to its integrated Wi-Fi capability and sufficient processing power for handling web servers and LoRa communication simultaneously.

Step 2 - Hardware Interfacing
SPI communication is used between ESP32 and SX1278:

SX1278 Pin	ESP32 GPIO
NSS	GPIO 5
RST	GPIO 14
DIO0	GPIO 2
MOSI	GPIO 23
MISO	GPIO 19
SCK	GPIO 18

OLED display connections:

OLED Pin	ESP32 GPIO
SDA	GPIO 21

Step 3 - Firmware Development
The firmware implements:

- LoRa packet transmission and reception
- Embedded HTTP web server
- ACK-based packet confirmation
- OLED monitoring

- Wi-Fi Access Point creation
- Real-time JSON status updates

4.2 Software Design Flow

Step 4 - Web Interface

A responsive mobile-optimized HTML/CSS/JavaScript interface is stored directly inside the ESP32 program memory using PROGMEM.

The interface includes:

- Real-time chat viewport
- Signal monitoring
- Message counters
- Dynamic message rendering
- Auto-refresh updates

Step 5 - Communication Protocol

The communication protocol uses:

- Message prefixes (ALPHA/DELTA)
- ACK packets
- RSSI monitoring
- Packet synchronization
- CRC error checking

Step 6 - Real-Time Status Updates

The browser continuously fetches communication data from the ESP32 using: /status

JSON responses contain:

- Message count
- RSSI values
- Message history
- Delivery status

5. Circuit Diagram

5.1 Pin Connection Table

Components	ESP32 GPIO
LoRa NSS	GPIO 5
LoRa RST	GPIO 14
LoRa DIO0	GPIO 2
OLED SDA	GPIO 21
OLED SCL	GPIO 22

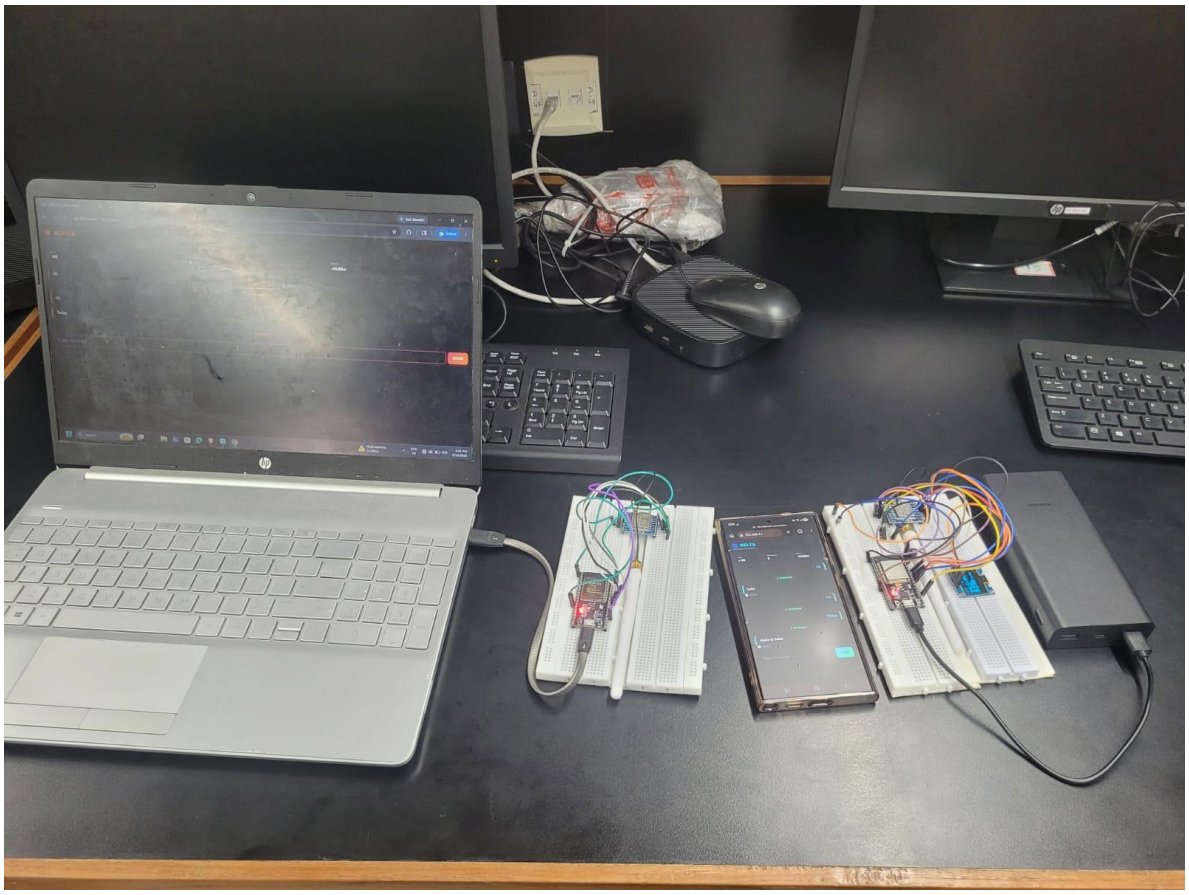


Figure 5.1: ESP32 + SX1278 LoRa Hardware Circuit Setup

5.2 Hardware Setup

The hardware setup consists of:

- Two ESP32 boards
- Two SX1278 LoRa modules
- One SSD1306 OLED display
- Smartphone/laptop browser interface

The ESP32 creates a Wi-Fi hotspot with:
SSID: Romeo
Password: Sierra

Users connect to this hotspot and access the web interface hosted by the ESP32.

Similarly, the receiver node (DELTA) creates a separate Wi-Fi hotspot with:
SSID: Sierra
Password: Romeo

This allows users to independently access the receiver node interface for monitoring received messages, communication status, and system diagnostics.

Both ESP32 nodes are powered through USB supply and communicate wirelessly using SX1278 LoRa transceivers operating at 433 MHz. The OLED display is connected to the ESP32 through the I2C interface for real-time monitoring and status visualization. Proper antenna connection is maintained on both LoRa modules to ensure stable long-range communication performance.

The compact hardware design makes the system portable and suitable for laboratory experiments, field testing, and standalone wireless communication applications. The modular architecture also allows easy expansion for future improvements such as sensor integration, encryption, and multi-node communication networks.

6. Working of the Model

6.1 Transmitter Node Operation

The transmitter node performs the following operations:

- Creates Wi-Fi Access Point
- Hosts web server on port 80
- Accepts user messages through browser
- Transmits LoRa packets
- Waits for acknowledgement packets
- Updates OLED display
- Tracks RSSI values

6.2 Receiver Node Operation

The receiver node continuously:

- Monitors incoming LoRa packets
- Extracts transmitted messages
- Updates RSSI values
- Sends acknowledgement packets
- Displays communication status on OLE

6.3 System Architecture

The complete system hardware interfacing is shown below:

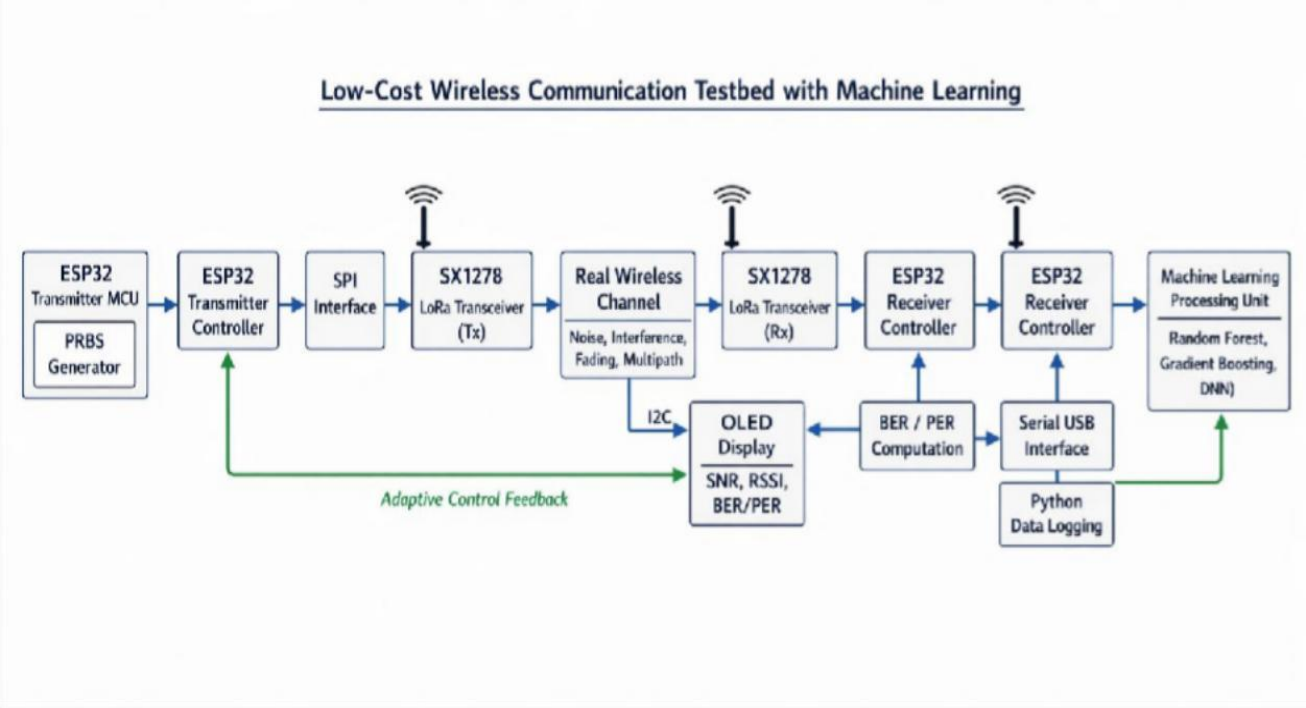


Figure 6.1: System Architecture — Transmitter and Receiver Node Interfacing

6.4 Data Flow

The complete data flow is:
User Message → Web Interface → ESP32 → LoRa Transmission → Receiver Node → ACK Response → OLED/Web Update.

- At each stage of this pipeline, the data is processed and validated to ensure accuracy and reliability. The ESP32 handles both HTTP request parsing and LoRa packet formatting, acting as a bridge between the web interface and wireless communication layer.
- Once the receiver node obtains the packet, it immediately triggers acknowledgment generation while simultaneously updating local display and system logs. This ensures minimal delay between reception and confirmation.
- The bidirectional flow allows continuous monitoring of link quality, message integrity, and system responsiveness, making the overall communication process efficient and dependable even in unstable wireless conditions.
- At each stage of this pipeline, the data is processed and validated to ensure accuracy and reliability. The ESP32 handles both HTTP request parsing and LoRa packet formatting, acting as a bridge between the web interface and wireless communication layer.
- Once the receiver node obtains the packet, it immediately triggers acknowledgment generation while simultaneously updating local display and system logs. This ensures minimal delay between reception and confirmation.
- The bidirectional flow allows continuous monitoring of link quality, message integrity, and system responsiveness, making the overall communication process efficient and dependable even in unstable wireless conditions.
- The integration of both web and LoRa layers ensures that users can interact with the system in a simple and intuitive manner while the underlying wireless communication remains highly efficient. This separation of user interface and communication protocol improves system scalability and maintainability.
- Furthermore, the system architecture supports future enhancements such as multi-node networking, encryption of transmitted data, and integration with cloud logging when internet access is available. These features can be added without modifying the core communication workflow.
- Thus, the complete data flow not only ensures reliable message delivery but also provides a flexible foundation for developing advanced low-power wireless communication systems.

7. Results

The system successfully demonstrated stable two-way LoRa communication between ESP32 nodes using the SX1278 transceiver modules.

7.1 Communication Performance

The implemented system successfully achieved:

- Real-time bidirectional communication
- ACK-based delivery confirmation
- Stable web server operation
- Real-time OLED monitoring
- RSSI-based signal tracking
- Mobile-responsive messaging interface

7.2 RSSI Monitoring

The system continuously monitored signal strength using RSSI values measured directly from the LoRa hardware.

Typical observed values:

RSSI	Signal Quality
-40 dBm	Excellent
-70 dBm	Good
-100 dBm	Weak

7.3 Web Interface Performance

The mobile-optimized web interface successfully:

- Displayed real-time messages
- Updated delivery status dynamically
- Refreshed communication metrics every second
- Maintained responsive operation across devices

7.4 OLED Monitoring

The OLED display successfully provided:

- Message counters
- RSSI display
- Sent/received indicators
- ACK confirmation
- Recent communication history

8. Applications

The proposed ESP32-based two-way LoRa communication system with real-time web interface and OLED monitoring has several practical applications in wireless communication and IoT domains.

8.1 Emergency Communication

The system can be effectively used in emergency situations where traditional communication networks are unavailable or fail. Since it operates without internet dependency, it ensures continuous message exchange between users in critical scenarios.

8.2 Disaster Management

During natural disasters such as floods, earthquakes, or cyclones, the system can be deployed as a portable communication network. Its long-range LoRa capability allows quick establishment of communication links between rescue teams and control centers.

8.3 Remote Area Communication

The system is suitable for rural, forest, and mountainous regions where cellular coverage is weak or unavailable. It enables reliable long-distance messaging between isolated locations.

8.4 IoT Messaging Systems

The architecture can be extended for IoT-based applications such as remote device control, sensor data communication, and smart monitoring systems. It can act as a lightweight command-and-control platform for distributed IoT nodes.

8.5 Educational Wireless Research

The system provides a low-cost and practical platform for students and researchers to study LoRa communication, embedded system design, web-based interfaces, and wireless networking protocols in real-time environments.

8.6 Military and Security Communication

The system can be adapted for secure field communication in defense and security operations where infrastructure-based networks are not reliable. Its long-range and low-power nature makes it suitable for tactical communication setups.

8.7 Industrial Monitoring Systems

It can be used in industries for machine-to-machine communication, equipment status updates, and remote monitoring in large-scale facilities such as factories, warehouses, and plants.

8.8 Smart Agriculture

The system can support agricultural applications by enabling communication between distributed sensor nodes for monitoring soil conditions, irrigation status, and farm equipment in remote fields.

8.9 Campus or Local Network Communication

The system can be deployed in educational institutions or campuses to create a private communication network for messaging, announcements, or internal coordination without relying on internet services

9. Future Scope and Conclusion

9.1 Future Scope

The proposed system can be extended further using:

- AES encryption for secure messaging
- Multi-node mesh networking
- GPS integration
- Voice packet transmission
- SD card message logging
- Battery-powered portable nodes
- LoRaWAN cloud integration
- Mobile application support
- End-to-end encryption

9.2 Conclusion

This project successfully demonstrates a low-cost ESP32-based two-way LoRa communication system integrated with a browser-based user interface and OLED monitoring system. The implementation combines LoRa wireless communication, Wi-Fi networking, web technologies, and embedded systems into a single standalone communication platform.

The ACK-based message confirmation mechanism improves reliability while RSSI monitoring enables real-time signal diagnostics. The system achieves stable long-range communication using the SX1278 LoRa transceiver operating at 433 MHz.

The project demonstrates the practical integration of embedded web servers and LoRa PHY communication, making it suitable for emergency communication, IoT applications, educational experimentation, and remote messaging systems.
The proposed architecture provides a scalable foundation for future wireless communication research and real-world deployment.

10. References

[1] Semtech Corporation, *SX1276/77/78/79 - LoRa Modem Designer's Guide*, Semtech Corporation, 2020.
Available: https://www.semtech.com/

[2] Espressif Systems, *ESP32 Series Datasheet*, Version 3.4, Espressif Systems, 2022.
Available: https://www.espressif.com/

[3] Sandeep Mistry, *Arduino LoRa Library*, GitHub Repository.
Available: https://github.com/sandeepmistry/arduino-LoRa

[4] Adafruit Industries, *Adafruit SSD1306 OLED Display Library Documentation*, GitHub Repository.
Available: https://github.com/adafruit/Adafruit_SSD1306

[5] Adafruit Industries, *Adafruit GFX Graphics Library*, GitHub Repository.
Available: https://github.com/adafruit/Adafruit-GFX-Library

[6] Arduino Documentation, *ESP32 Web Server and Wi-Fi Programming Guide*, Arduino Official Documentation.
Available: https://docs.arduino.cc/

[7] LoRa Alliance, *LoRaWAN Specification v1.0.4*, LoRa Alliance Technical Committee, 2020.
Available: https://lora-alliance.org/

[8] N. Sornin, M. Luis, T. Eirich, T. Kramp, and O. Hersent, "LoRaWAN Specification," *LoRa Alliance*, Jan. 2015.

[9] U. Raza, P. Kulkarni, and M. Sooriyabandara, "Low Power Wide Area Networks: An Overview," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 855–873, 2017.

[10] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, "Understanding the Limits of LoRaWAN," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[11] A. Augustin, J. Yi, T. Clausen, and W. Townsley, "A Study of LoRa: Long Range & Low Power Networks for the Internet of Things," *Sensors*, vol. 16, no. 9, 2016.

[12] K. Mekki, E. Bajic, F. Chaxel, and F. Meyer, "A Comparative Study of LPWAN Technologies for Large-Scale IoT Deployment," *ICT Express*, vol. 5, no. 1, pp. 1–7, 2019.

[13] M. Centenaro, L. Vangelista, A. Zanella, and M. Zorzi, "Long-Range Communications in Unlicensed Bands: The Rising Stars in the IoT and Smart City Scenarios," *IEEE Wireless Communications*, vol. 23, no. 5, pp. 60–67, 2016.

[14] Random Nerd Tutorials, *ESP32 LoRa Communication Tutorial*.
Available: https://randomnerdtutorials.com/

[15] OLED Display SSD1306 Datasheet, Solomon Systech Limited, 2008.
Available: https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

11. Appendix

Transmitter Arduino IDE Code:

```
#include <WiFi.h>
#include <WebServer.h>
#include <LoRa.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <nvs_flash.h>

#define PIN_SS   5
#define PIN_RST  14
#define PIN_DIO0 2
#define LORA_FREQ 433E6

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_SDA 21
#define OLED_SCL 22
#define OLED_ADDR 0x3C

const char* txSsid = "Romeo";
const char* txPassword = "Sierra";

WebServer txServer(80);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

struct Message {
  String text;
  bool ackReceived;
  bool isReceived;
};
Message messages[10];
int msgIndex = 0;
int lastRssi = 0;
bool oledAvailable = false;

void updateOLED() {
  if (!oledAvailable) return;
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0,0);
  display.println("ALPHA // TWO-WAY");
  display.drawLine(0,10,128,10,WHITE);
  display.setCursor(0,15);
```

```
display.printf("Total: %d | RSSI: %d\n", msgIndex, lastRssi);
  display.drawLine(0,25,128,25,WHITE);

  int y = 30;
  for (int i = max(0, msgIndex - 3); i < msgIndex; i++) {
    display.setCursor(0, y);
    char prefix = messages[i].isReceived ? 'R' : 'S';
    char status = messages[i].ackReceived ? 'OK' : '..';
    display.printf("[%c,%c] %s\n", prefix, status, messages[i].text.substring(0, 12).c_str());
    y += 10;
  }
  display.display();
}

// ═══════════════════════════════════════════════════════════════
// TWO-WAY MOBILE-OPTIMIZED UI
// ═══════════════════════════════════════════════════════════════
const char WEB_PAGE[] PROGMEM = R"=====(
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no, viewport-fit=cover">
<title>ALPHA // TWO-WAY</title>
<style>
  :root {
    --bg: #060305;
    --surface: #150a11;
    --surface-border: #2c1623;
    --crimson: #ff2a54;
    --crimson-glow: rgba(255, 42, 84, 0.4);
    --orange: #ff7300;
    --text-main: #fdf5f7;
    --text-dim: #9e7f89;
  }

  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    background-color: var(--bg);
    color: var(--text-main);
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
    display: flex;
    flex-direction: column;
    height: 100vh;
    height: -webkit-fill-available;
    overflow: hidden;
  }

  .header {
```

```
background: linear-gradient(180deg, #1b0a13 0%, var(--bg) 100%);
  border-bottom: 1px solid var(--surface-border);
  padding: 14px 16px;
  flex-shrink: 0;
}

.title {
  font-size: 20px; font-weight: 900; color: var(--crimson);
  letter-spacing: 1.5px; text-shadow: 0 0 10px var(--crimson-glow);
}
.subtitle { font-size: 10px; color: var(--text-dim); margin-top: 2px; }

.metrics {
  display: flex; gap: 8px; padding: 10px 16px;
  background: var(--surface); border-bottom: 1px solid var(--surface-border);
  flex-shrink: 0;
}
.metric-card {
  flex: 1; background: rgba(5, 2, 4, 0.6);
  border: 1px solid var(--surface-border); border-radius: 6px;
  padding: 6px 10px;
}
.metric-lbl { font-size: 9px; color: var(--text-dim); font-weight: 700; }
.metric-val { font-size: 14px; font-weight: 800; color: #fff; margin-top: 2px; }

.status-dot {
  width: 6px; height: 6px; border-radius: 50%; background: var(--orange);
  box-shadow: 0 0 8px var(--orange); animation: pulse 2s infinite;
  display: inline-block; margin-right: 4px;
}
@keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.3; } }

.chat-viewport {
  flex: 1; overflow-y: auto; padding: 12px 16px;
  display: flex; flex-direction: column; gap: 10px;
  -webkit-overflow-scrolling: touch;
}

.msg-container { display: flex; flex-direction: column; max-width: 85%; animation: fadeIn 0.3s ease-out; }
@keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY(0); } }

.msg-container.rx { align-self: flex-start; }
.msg-container.tx { align-self: flex-end; align-items: flex-end; }
.msg-container.sys { align-self: center; max-width: 90%; }

.bubble {
  padding: 10px 14px; border-radius: 12px; font-size: 14px;
  line-height: 1.4; word-break: break-word; font-weight: 500;
  box-shadow: 0 4px 12px rgba(0,0,0,0.2);
}
```

```
  background: linear-gradient(180deg, #1b0a13 0%, var(--bg) 100%);
  border-bottom: 1px solid var(--surface-border);
  padding: 14px 16px;
  flex-shrink: 0;
}

.title {
  font-size: 20px; font-weight: 900; color: var(--crimson);
  letter-spacing: 1.5px; text-shadow: 0 0 10px var(--crimson-glow);
}
.subtitle { font-size: 10px; color: var(--text-dim); margin-top: 2px; }

.metrics {
  display: flex; gap: 8px; padding: 10px 16px;
  background: var(--surface); border-bottom: 1px solid var(--surface-border);
  flex-shrink: 0;
}
.metric-card {
  flex: 1; background: rgba(5, 2, 4, 0.6);
  border: 1px solid var(--surface-border); border-radius: 6px;
  padding: 6px 10px;
}
.metric-lbl { font-size: 9px; color: var(--text-dim); font-weight: 700; }
.metric-val { font-size: 14px; font-weight: 800; color: #fff; margin-top: 2px; }

.status-dot {
  width: 6px; height: 6px; border-radius: 50%; background: var(--orange);
  box-shadow: 0 0 8px var(--orange); animation: pulse 2s infinite;
  display: inline-block; margin-right: 4px;
}
@keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.3; } }

.chat-viewport {
  flex: 1; overflow-y: auto; padding: 12px 16px;
  display: flex; flex-direction: column; gap: 10px;
  -webkit-overflow-scrolling: touch;
}

.msg-container { display: flex; flex-direction: column; max-width: 85%; animation: fadeIn 0.3s ease-out; }
@keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY(0); } }

.msg-container.rx { align-self: flex-start; }
.msg-container.tx { align-self: flex-end; align-items: flex-end; }
.msg-container.sys { align-self: center; max-width: 90%; }

.bubble {
  padding: 10px 14px; border-radius: 12px; font-size: 14px;
  line-height: 1.4; word-break: break-word; font-weight: 500;
  box-shadow: 0 4px 12px rgba(0,0,0,0.2);
}
```

```
rx .bubble {
    background: var(--surface); border: 1px solid var(--surface-border);
    border-left: 4px solid var(--crimson); border-bottom-left-radius: 4px;
  }
  .tx .bubble {
    background: linear-gradient(135deg, #1a0c12 0%, #0f060a 100%);
    border: 1px solid rgba(255, 42, 84, 0.3); border-right: 4px solid var(--orange);
    border-bottom-right-radius: 4px; color: var(--crimson);
  }
  .sys .bubble {
    background: rgba(255, 115, 0, 0.1); border: 1px solid rgba(255, 115, 0, 0.2);
    border-radius: 20px; padding: 4px 12px; font-size: 11px; color: var(--orange);
  }

  .meta { font-size: 10px; color: var(--text-dim); margin-top: 2px; padding: 0 4px; }

  .controller {
    background: #080306; border-top: 1px solid var(--surface-border);
    padding: 10px 16px; padding-bottom: calc(10px + env(safe-area-inset-bottom));
    display: flex; gap: 8px; flex-shrink: 0;
  }

  .input-core {
    flex: 1; background: #030102; border: 1px solid var(--surface-border);
    border-radius: 8px; padding: 0 12px; color: #fff;
    font-size: 16px; height: 42px; outline: none; transition: border 0.2s;
  }
  .input-core:focus { border-color: var(--crimson); box-shadow: 0 0 10px var(--crimson-glow); }

  .send-core {
    background: linear-gradient(135deg, var(--crimson) 0%, var(--orange) 100%);
    color: #fff; font-size: 13px; font-weight: 800; border: none;
    border-radius: 8px; padding: 0 16px; height: 42px;
    cursor: pointer; display: flex; align-items: center;
    box-shadow: 0 4px 15px var(--crimson-glow);
  }
  .send-core:active { opacity: 0.7; }
  .send-core:disabled { opacity: 0.4; }

  .empty { text-align: center; color: var(--text-dim); padding: 20px; }
</style>
</head>
<body>

<div class="header">
  <div class="title">🟥 ALPHA</div>
  <div class="subtitle">Two-Way Link Active</div>
</div>

<div class="metrics">
  <div class="metric-card">
    <div class="metric-lbl">Link</div>
```

```
    <div class="metric-val"><span class="status-dot" id="dot"></span><span id="st">OK</span></div>
  </div>
  <div class="metric-card">
    <div class="metric-lbl">Messages</div>
    <div class="metric-val" id="mc">0</div>
  </div>
  <div class="metric-card">
    <div class="metric-lbl">Signal</div>
    <div class="metric-val" id="rs">--</div>
  </div>
</div>

<div class="chat-viewport" id="chat"><div class="empty">Waiting for messages...</div></div>

<div class="controller">
  <input type="text" id="inp" class="input-core" placeholder="Type message..." maxlength="200" autocomplete="off">
  <button onclick="send()" class="send-core" id="btn">SEND</button>
</div>

<script>
let lastC = 0; let busy = false;

function send() {
  if (busy) return;
  const f = document.getElementById("inp"); const v = f.value.trim(); if (!v) return;
  busy = true; document.getElementById("btn").disabled = true;
  fetch("/send?msg=" + encodeURIComponent(v)).then(() => { f.value = ""; update(); }).finally(() => { busy = false;
document.getElementById("btn").disabled = false; });
}
document.getElementById("inp").onkeydown = e => { if (e.key === "Enter") send(); };

function update() {
  fetch("/status").then(r => r.json()).then(d => {
    document.getElementById("mc").innerText = d.msgCount;
    document.getElementById("rs").innerText = d.rssi ? d.rssi + "dBm" : "--";
    document.getElementById("st").innerText = "OK";
    document.getElementById("dot").style.background = "var(--orange)";
    if (d.msgCount !== lastC) { lastC = d.msgCount; render(d.messages); }
  }).catch(() => { document.getElementById("st").innerText = "ERROR"; document.getElementById("dot").style.background
= "#ff0033"; });
}

function render(arr) {
  const v = document.getElementById("chat");
  if (arr.length === 0) { v.innerHTML = '<div class="empty">No messages yet</div>'; return; }
  v.innerHTML = "";
  arr.forEach(m => {
    const d = document.createElement("div");
    if (m.ackReceived) {
      d.className = "msg-container sys";
      d.innerHTML = `<div class="bubble">✓ Delivered</div>`;
```

```
    } else if (m.isReceived) {
      d.className = "msg-container rx";
      d.innerHTML = `<div class="bubble">${m.text}</div>`;
    } else {
      d.className = "msg-container tx";
      d.innerHTML = `<div class="bubble">${m.text}</div>`;
    }
    v.appendChild(d);
  });
  v.scrollTo({ top: v.scrollHeight, behavior: 'smooth' });
}

setInterval(update, 1000); update();
</script>
</body>
</html>
)=====";

void sendMessage(String msg) {
  Serial.printf("[ALPHA] SENDING: %s\n", msg.c_str());
  LoRa.beginPacket();
  LoRa.print("ALPHA:" + msg);
  LoRa.endPacket();

  if (msgIndex >= 10) {
    for (int i = 0; i < 9; i++) messages[i] = messages[i+1];
    msgIndex = 9;
  }
  messages[msgIndex].text = msg;
  messages[msgIndex].ackReceived = false;
  messages[msgIndex].isReceived = false;
  msgIndex++;

  unsigned long st = millis();
  while (millis() - st < 2500) {
    if (LoRa.parsePacket()) {
      String a = "";
      while (LoRa.available()) a += (char)LoRa.read();
      if (a == "ACK:ALPHA:" + msg) {
        messages[msgIndex-1].ackReceived = true;
        lastRssi = LoRa.packetRssi();
        Serial.println("[ALPHA] ACK received");
        break;
      }
    }
    delay(10);
  }
  updateOLED();
}

void handleRoot() { txServer.send_P(200, "text/html", WEB_PAGE); }
```

```
void handleSend() {
  if (txServer.hasArg("msg")) {
    String m = txServer.arg("msg");
    m.trim();
    if (m.length() > 0) {
      sendMessage(m);
      txServer.send(200, "text/plain", "OK");
      return;
    }
  }
  txServer.send(400, "text/plain", "ERR");
}

void handleStatus() {
  String j = "{\"msgCount\":" + String(msgIndex) + ",\"rssi\":" + String(lastRssi) + ",\"messages\":[";
  for (int i = 0; i < msgIndex; i++) {
    if (i > 0) j += ",";
    String t = messages[i].text;
    t.replace("\\","\\\\");
    t.replace("\"","\\\"");
    j += "{\"text\":\"" + t + "\",\"ackReceived\":" + String(messages[i].ackReceived ? "true" : "false") + ",\"isReceived\":" +
String(messages[i].isReceived ? "true" : "false") + "}";
  }
  j += "]}";
  txServer.send(200, "application/json", j);
}

void setup() {
  Serial.begin(115200);
  Wire.begin(OLED_SDA, OLED_SCL);
  if (display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) oledAvailable = true;

  nvs_flash_erase();
  nvs_flash_init();
  WiFi.persistent(false);
  WiFi.disconnect(true, true);
  WiFi.softAPdisconnect(true);
  WiFi.mode(WIFI_OFF);
  delay(200);

  Serial.println("\n[ALPHA] Compile: " __DATE__ " " __TIME__);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(txSsid, txPassword, 1, 0, 4);

  LoRa.setPins(PIN_SS, PIN_RST, PIN_DIO0);
  if (!LoRa.begin(LORA_FREQ)) {
    Serial.println("[ERROR] LoRa failed");
    while (1);
  }
  LoRa.setSpreadingFactor(7);
```

```
LoRa.setSignalBandwidth(500E3);
  LoRa.setCodingRate4(5);
  LoRa.setSyncWord(0xA5);
  LoRa.enableCrc();

  txServer.on("/", handleRoot);
  txServer.on("/send", handleSend);
  txServer.on("/status", handleStatus);
  txServer.begin();

  if (oledAvailable) updateOLED();

  Serial.println("[ALPHA] READY - SSID: Romeo / PASS: Sierra");
}

void loop() {
  txServer.handleClient();

  int sz = LoRa.parsePacket();
  if (sz) {
    String m = "";
    while (LoRa.available()) m += (char)LoRa.read();
    int r = LoRa.packetRssi();

    Serial.printf("[ALPHA] RECEIVED: %s\n", m.c_str());

    // Message FROM DELTA
    if (m.startsWith("DELTA:")) {
      String text = m.substring(6);

      // Add to message list
      if (msgIndex >= 10) {
        for (int i = 0; i < 9; i++) messages[i] = messages[i+1];
        msgIndex = 9;
      }
      messages[msgIndex].text = text;
      messages[msgIndex].ackReceived = false;
      messages[msgIndex].isReceived = true;
      msgIndex++;

      lastRssi = r;

      // Send ACK
      delay(150);
      LoRa.beginPacket();
      LoRa.print("ACK:DELTA:" + text);
      LoRa.endPacket();
      Serial.println("[ALPHA] ACK sent");
      updateOLED();
    }
  }
}
```

Receiver Arduino IDE Code:

```
#include <WiFi.h>
 #include <WebServer.h>
 #include <LoRa.h>
 #include <nvs_flash.h>

 #define PIN_SS   5
 #define PIN_RST  14
 #define PIN_DIO0 2
 #define LORA_FREQ 433E6

 const char* ssid = "Sierra";
 const char* password = "Romeo";

 WebServer server(80);

 struct Message {
  String text;
  int rssi;
  float snr;
  String time;
  bool isAck;
  bool isSent;
 };

 Message messages[50];
 int msgCount = 0;

 String escapeJson(String s) {
  s.replace("\\", "\\\\");
  s.replace("\"", "\\\"");
  s.replace("\n", "\\n");
  s.replace("\r", "\\r");
  return s;
 }

 String getUptime() {
  unsigned long seconds = millis() / 1000;
  unsigned long minutes = seconds / 60;
  unsigned long hours = minutes / 60;
  seconds %= 60;
  minutes %= 60;
  char buf[20];
  sprintf(buf, "%02lu:%02lu:%02lu", hours, minutes, seconds);
  return String(buf);
 }
```

```
void addMessage(String text, int rssi, float snr, bool isAck, bool isSent) {
   if (msgCount >= 50) {
     for (int i = 0; i < 49; i++) messages[i] = messages[i + 1];
     msgCount = 49;
   }
   messages[msgCount].text = text;
   messages[msgCount].rssi = rssi;
   messages[msgCount].snr = snr;
   messages[msgCount].time = getUptime();
   messages[msgCount].isAck = isAck;
   messages[msgCount].isSent = isSent;
   msgCount++;
   Serial.printf("[DELTA] +MSG: %s (Total: %d)\n", text.c_str(), msgCount);
 }

 // ═══════════════════════════════════════════════════════════════════════
 // TWO-WAY MOBILE-OPTIMIZED UI
 // ═══════════════════════════════════════════════════════════════════════
 const char WEBPAGE[] PROGMEM = R"=====(
 <!DOCTYPE html>
 <html lang="en">
 <head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no, viewport-
fit=cover">
 <title>DELTA // TWO-WAY</title>
 <style>
   :root {
     --bg: #030509;
     --surface: #0a0f1d;
     --surface-border: #131d38;
     --cyan: #00f3ff;
     --cyan-glow: rgba(0, 243, 255, 0.4);
     --green: #00ffaa;
     --text-main: #f0f6fc;
     --text-dim: #798da8;
   }

   * { box-sizing: border-box; margin: 0; padding: 0; }

   body {
     background-color: var(--bg);
     color: var(--text-main);
     font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
     display: flex;
     flex-direction: column;
     height: 100vh;
     height: -webkit-fill-available;
     overflow: hidden;
   }
```

```
.header {
    background: linear-gradient(180deg, #091021 0%, var(--bg) 100%);
    border-bottom: 1px solid var(--surface-border);
    padding: 14px 16px;
    flex-shrink: 0;
  }

  .title {
    font-size: 20px; font-weight: 900; color: var(--cyan);
    letter-spacing: 1.5px; text-shadow: 0 0 10px var(--cyan-glow);
  }
  .subtitle { font-size: 10px; color: var(--text-dim); margin-top: 2px; }

  .metrics {
    display: flex; gap: 8px; padding: 10px 16px;
    background: var(--surface); border-bottom: 1px solid var(--surface-border);
    flex-shrink: 0;
  }
  .metric-card {
    flex: 1; background: rgba(3, 5, 9, 0.6);
    border: 1px solid var(--surface-border); border-radius: 6px;
    padding: 6px 10px;
  }
  .metric-lbl { font-size: 9px; color: var(--text-dim); font-weight: 700; }
  .metric-val { font-size: 14px; font-weight: 800; color: #fff; margin-top: 2px; }

  .status-dot {
    width: 6px; height: 6px; border-radius: 50%; background: var(--green);
    box-shadow: 0 0 8px var(--green); animation: pulse 2s infinite;
    display: inline-block; margin-right: 4px;
  }
  @keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.3; } }

  .chat-viewport {
    flex: 1; overflow-y: auto; padding: 12px 16px;
    display: flex; flex-direction: column; gap: 10px;
    -webkit-overflow-scrolling: touch;
  }

  .msg-container { display: flex; flex-direction: column; max-width: 85%; animation: fadeIn 0.3s ease-out; }
  @keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY(0); } }

  .msg-container.rx { align-self: flex-start; }
  .msg-container.tx { align-self: flex-end; align-items: flex-end; }
  .msg-container.sys { align-self: center; max-width: 90%; }

  .bubble {
    padding: 10px 14px; border-radius: 12px; font-size: 14px;
    line-height: 1.4; word-break: break-word; font-weight: 500;
    box-shadow: 0 4px 12px rgba(0,0,0,0.2);
  }
```

```
  .rx .bubble {
    background: var(--surface); border: 1px solid var(--surface-border);
    border-left: 4px solid var(--cyan); border-bottom-left-radius: 4px;
  }
  .tx .bubble {
    background: linear-gradient(135deg, #091a2f 0%, #051321 100%);
    border: 1px solid rgba(0, 243, 255, 0.3); border-right: 4px solid var(--green);
    border-bottom-right-radius: 4px; color: var(--cyan);
  }
  .sys .bubble {
    background: rgba(0, 255, 170, 0.1); border: 1px solid rgba(0, 255, 170, 0.2);
    border-radius: 20px; padding: 4px 12px; font-size: 11px; color: var(--green);
  }

  .meta { font-size: 10px; color: var(--text-dim); margin-top: 2px; padding: 0 4px; }

  .controller {
    background: #050a14; border-top: 1px solid var(--surface-border);
    padding: 10px 16px; padding-bottom: calc(10px + env(safe-area-inset-bottom));
    display: flex; gap: 8px; flex-shrink: 0;
  }

  .input-core {
    flex: 1; background: #020408; border: 1px solid var(--surface-border);
    border-radius: 8px; padding: 0 12px; color: #fff;
    font-size: 16px; height: 42px; outline: none; transition: border 0.2s;
  }
  .input-core:focus { border-color: var(--cyan); box-shadow: 0 0 10px var(--cyan-glow); }

  .send-core {
    background: linear-gradient(135deg, var(--cyan) 0%, var(--green) 100%);
    color: #000; font-size: 13px; font-weight: 800; border: none;
    border-radius: 8px; padding: 0 16px; height: 42px;
    cursor: pointer; display: flex; align-items: center;
    box-shadow: 0 4px 15px var(--cyan-glow);
  }
  .send-core:active { opacity: 0.7; }
  .send-core:disabled { opacity: 0.4; }

  .empty { text-align: center; color: var(--text-dim); padding: 20px; }
</style>
</head>
<body>

<div class="header">
  <div class="title">🟢 DELTA</div>
  <div class="subtitle">Two-Way Link Active</div>
</div>
```

```
<div class="metrics">
   <div class="metric-card">
     <div class="metric-lbl">Link</div>
     <div class="metric-val"><span class="status-dot" id="dot"></span><span id="st">OK</span></div>
   </div>
   <div class="metric-card">
     <div class="metric-lbl">Messages</div>
     <div class="metric-val" id="mc">0</div>
   </div>
   <div class="metric-card">
     <div class="metric-lbl">Signal</div>
     <div class="metric-val" id="rs">--</div>
   </div>
 </div>

 <div class="chat-viewport" id="chat"><div class="empty">Waiting for messages...</div></div>

 <div class="controller">
   <input type="text" id="inp" class="input-core" placeholder="Type message..." maxlength="200" autocomplete="off">
   <button onclick="send()" class="send-core" id="btn">SEND</button>
 </div>

 <script>
 let lastC = 0; let busy = false;

 function send() {
   if (busy) return;
   const f = document.getElementById("inp"); const v = f.value.trim(); if (!v) return;
   busy = true; document.getElementById("btn").disabled = true;
   fetch("/send?msg=" + encodeURIComponent(v)).then(() => { f.value = ""; update(); }).finally(() => { busy = false;
document.getElementById("btn").disabled = false; });
 }
 document.getElementById("inp").onkeydown = e => { if (e.key === "Enter") send(); };

 function update() {
   fetch("/data").then(r => r.json()).then(d => {
     document.getElementById("mc").innerText = d.msgCount;
     document.getElementById("rs").innerText = d.lastRssi ? d.lastRssi + "dBm" : "--";
     document.getElementById("st").innerText = "OK";
     document.getElementById("dot").style.background = "var(--green)";
     if (d.msgCount !== lastC) { lastC = d.msgCount; render(d.messages); }
   }).catch(() => { document.getElementById("st").innerText = "ERROR"; document.getElementById("dot").style.background
= "#ff2a54"; });
 }

 function render(arr) {
   const v = document.getElementById("chat");
   if (arr.length === 0) { v.innerHTML = '<div class="empty">No messages yet</div>'; return; }
   v.innerHTML = "";
   arr.forEach(m => {
```

```
    const d = document.createElement("div");
    if (m.isAck) {
      d.className = "msg-container sys";
      d.innerHTML = `<div class="bubble">✓ Confirmed</div>`;
    } else if (m.isSent) {
      d.className = "msg-container tx";
      d.innerHTML = `<div class="bubble">${m.text}</div>`;
    } else {
      d.className = "msg-container rx";
      d.innerHTML = `<div class="bubble">${m.text}</div><div class="meta">📶 ${m.rssi}dBm</div>`;
    }
    v.appendChild(d);
  });
  v.scrollTo({ top: v.scrollHeight, behavior: 'smooth' });
}

setInterval(update, 1000); update();
</script>
</body>
</html>
)=====";

void handleRoot() { server.send_P(200, "text/html", WEBPAGE); }

void handleSend() {
  if (server.hasArg("msg")) {
    String m = server.arg("msg");
    m.trim();
    if (m.length() > 0) {
      Serial.printf("[DELTA] SENDING: %s\n", m.c_str());
      LoRa.beginPacket();
      LoRa.print("DELTA:" + m);
      LoRa.endPacket();
      addMessage(m, 0, 0, false, true);
      server.send(200, "text/plain", "OK");
      return;
    }
  }
  server.send(400, "text/plain", "ERR");
}

void handleData() {
  String j = "{\"msgCount\":" + String(msgCount) + ",\"lastRssi\":";
  int r = 0;
  for (int i = msgCount - 1; i >= 0; i--) {
    if (!messages[i].isSent && !messages[i].isAck) { r = messages[i].rssi; break; }
  }
  j += String(r) + ",\"messages\":[";
  for (int i = 0; i < msgCount; i++) {
    if (i > 0) j += ",";
```

```
    j += "{\"text\":\"" + escapeJson(messages[i].text) + "\",\"rssi\":" + String(messages[i].rssi) + ",\"isAck\":" + String(messages[i].isAck ? "true" : "false") + ",\"isSent\":" + String(messages[i].isSent ? "true" : "false") + "}";
   }
   j += "]}";
   server.send(200, "application/json", j);
  }

  void setup() {
   Serial.begin(115200);
   delay(1000);

   nvs_flash_erase();
   nvs_flash_init();
   WiFi.persistent(false);
   WiFi.disconnect(true, true);
   WiFi.softAPdisconnect(true);
   WiFi.mode(WIFI_OFF);
   delay(200);

   Serial.println("\n[DELTA] Compile: " __DATE__ " " __TIME__);

   WiFi.mode(WIFI_AP);
   WiFi.softAP(ssid, password, 1, 0, 4);

   server.on("/", handleRoot);
   server.on("/send", handleSend);
   server.on("/data", handleData);
   server.begin();

   LoRa.setPins(PIN_SS, PIN_RST, PIN_DIO0);
   if (!LoRa.begin(LORA_FREQ)) {
    Serial.println("[ERROR] LoRa failed");
    while (1);
   }
   LoRa.setSpreadingFactor(7);
   LoRa.setSignalBandwidth(500E3);
   LoRa.setCodingRate4(5);
   LoRa.setSyncWord(0xA5);
   LoRa.enableCrc();

   Serial.println("[DELTA] READY - SSID: Sierra / PASS: Romeo");
  }

  void loop() {
   server.handleClient();

   int sz = LoRa.parsePacket();
   if (sz) {
    String m = "";
```

```
    while (LoRa.available()) m += (char)LoRa.read();
    int r = LoRa.packetRssi();
    float s = LoRa.packetSnr();

    Serial.printf("[DELTA] RECEIVED: %s\n", m.c_str());

    // Message FROM ALPHA
    if (m.startsWith("ALPHA:")) {
      String text = m.substring(6);
      addMessage(text, r, s, false, false);

      // Send ACK
      delay(150);
      LoRa.beginPacket();
      LoRa.print("ACK:ALPHA:" + text);
      LoRa.endPacket();
      Serial.println("[DELTA] ACK sent");
    }
    // ACK for our message
    else if (m.startsWith("ACK:DELTA:")) {
      addMessage("", r, s, true, false);
    }
  }
}
```